Inteligencia Artificial

ENRIQUE PELÁEZ Ph.D.

Escuela Superior Politécnica del Litoral – ESPOL

Facultad de Ingeniería en Electricidad y Computación – FIEC

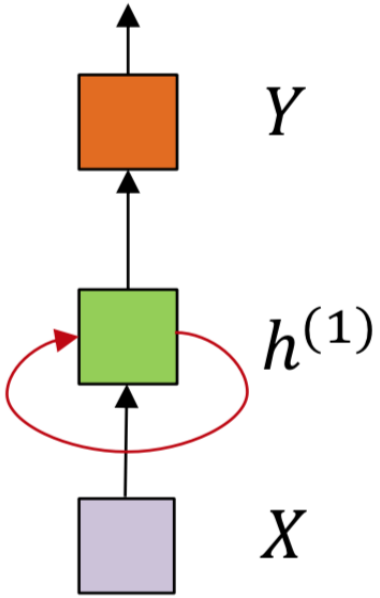
Clase # 16 - PAO 2026 – I

Agenda

1. Revisión del examen
2. Dónde estamos?
3. Redes neuronales recurrentes – RNN: Continuación
   3.1 Introducción al NLP – modelado del lenguaje
   3.2 Tokenizer, TF, TF-IDF, Embeddings
4. Un ejemplo

Redes neuronales recurrentes

- Ecuaciones:



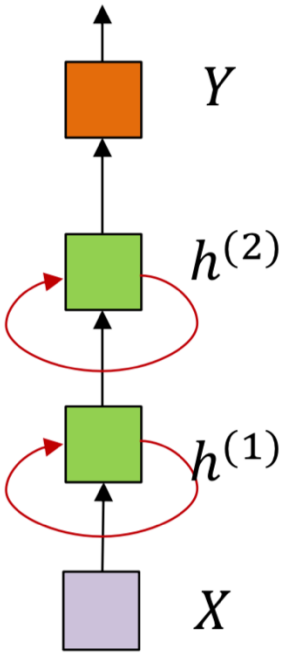
$$Y(t) = f_2\left(\sum_j w_{jk}^{(12)} h_j^{(1)}(t) + b_k^{(1)}, k = 1 \dots M\right)$$

$$h_i^{(1)}(t) = f_1\left(\sum_j w_{ji}^{(01)} X_j(t) + \sum_j w_{ji}^{(11)} h_i^{(1)}(t-1) + b_i^{(1)}\right)$$

- Notar que el superscript indica la capa de la red desde donde proviene la entrada
- Asumiendo la función de salida $f_2()$ - Softmax
- La función de activación en los nodos de la capa escondida $f_1()$ generalmente es *tanh()*
- Recordar cada neurona también tiene como entrada un *bias*

Redes neuronales recurrentes

- Ecuaciones:



$$Y(t) = f_3\left(\sum_j w_{jk}^{(23)} h_j^{(2)}(t) + b_k^{(3)}, k = 1 \dots M\right)$$

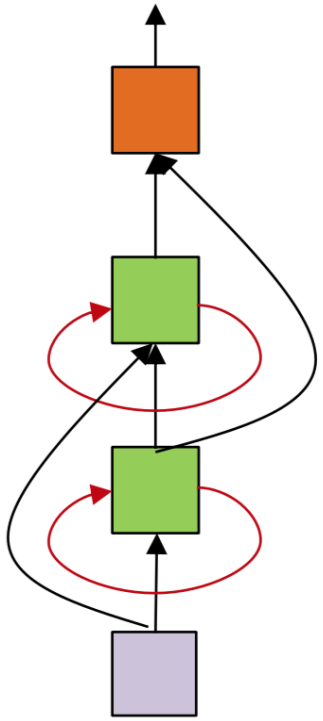
$$h_i^{(2)}(t) = f_2\left(\sum_j w_{ji}^{(12)} h_j^{(1)}(t) + \sum_j w_{ji}^{(22)} h_i^{(2)}(t-1) + b_i^{(2)}\right)$$

$$h_i^{(1)}(t) = f_1\left(\sum_j w_{ji}^{(01)} X_j(t) + \sum_j w_{ji}^{(11)} h_i^{(1)}(t-1) + b_i^{(1)}\right)$$

- Asumiendo la función de salida $f_3()$ - Softmax
- La función de activación en los nodos de la capa escondida $f_k()$ generalmente es *tanh()*
- Recordar cada neurona también tiene como entrada un *bias*

Redes neuronales recurrentes

- Ecuaciones:



$$Y(t) = f_3\left(\sum_j w_{jk}^{(23)} h_j^{(2)}(t) + \sum_j w_{ji}^{(13)} h_i^{(1)}(t) + b_k^{(3)}, k = 1 \dots M\right)$$

$$h_i^{(2)}(t) = f_2\left(\sum_j w_{ji}^{(12)} h_j^{(1)}(t) + \sum_j w_{ji}^{(02)} X_j(t) + \sum_j w_{ji}^{(22)} h_i^{(2)}(t-1) + b_i^{(2)}\right)$$

$$h_i^{(1)}(t) = f_1\left(\sum_j w_{ji}^{(01)} X_j(t) + \sum_j w_{ji}^{(11)} h_i^{(1)}(t-1) + b_i^{(1)}\right)$$

Redes neuronales recurrentes

Modelado del lenguaje

- En el modelado del lenguaje con *n-grams*, recopilamos la frecuencia de uso (estadística) de los diferentes *n-grams* en el texto, y usamos esa frecuencia para predecir la siguiente palabra.
- Un reto que enfrentan los modelos de lenguaje con *n-grams* es "sparsity", en particular cuando no hay suficientes datos en un **corpus o diccionario** para modelar el lenguaje con precisión (especialmente cuando *n* aumenta).
- Hay varios métodos de representación que mejoran esta debilidad: Bag-of-words - BoW, word embeddings, binary encoding, one-hot-encoding, TF, TF-IDF, latent semantic analysis, Word2Vec embedding….

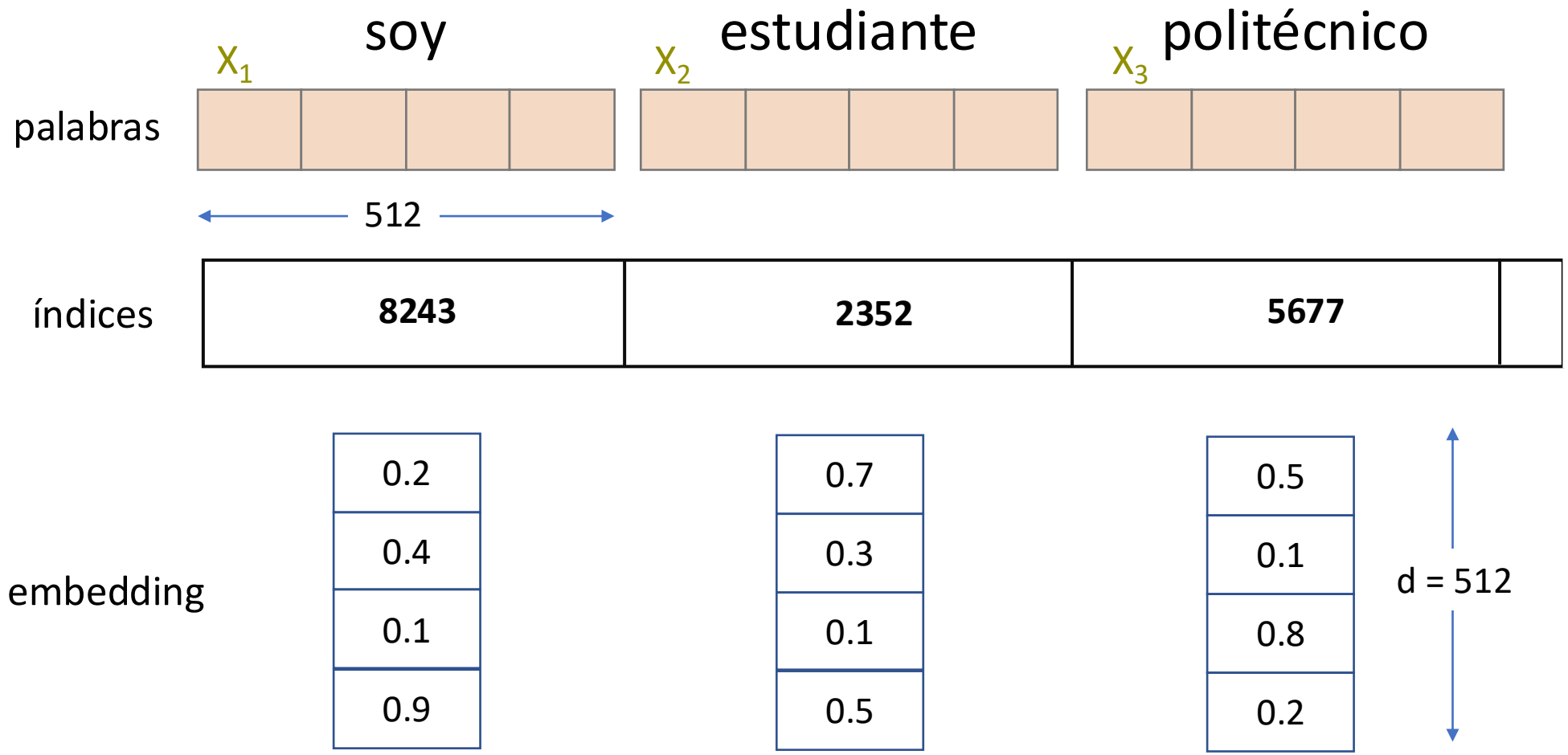
Redes neuronales recurrentes: Embeddings



Qué es un embedding?

- Es una representación numérica del texto, donde las palabras con igual significado tienen una representación similar
- Es un vector d-dimensional para cada número o índice de representación de cada palabra
- Si tenemos un diccionario de 10000 palabras, necesitamos 10000 índices
- Cada palabra tiene un índice único y cada índice tiene un vector embedding

Redes neuronales recurrentes: Embeddings



Redes neuronales recurrentes: Embeddings

TF-IDF

TF

- Frecuencia de términos: # de veces un término aparece en un documento en comparación con el # total de palabras en el documento.

$$TF = \frac{\text{\# de veces un término aparece en un documento}}{\text{\# total de palabras en el documento}}$$

IDF

- Frecuencia inversa de documentos: Proporción de documentos en el corpus que contienen el término.

$$\text{ID}F = \log\left(\frac{\text{\# de documentos en el corpus}}{\text{\# de documentos en el corpus que contienen el término}}\right)$$

$$\boldsymbol{TF.IDF = TF * IDF}$$

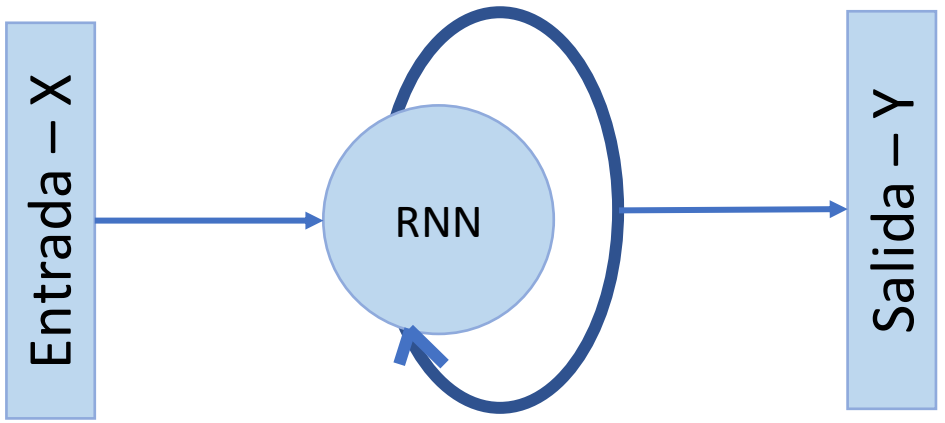
Redes neuronales recurrentes: Embeddings

Tipos de Embeddings:

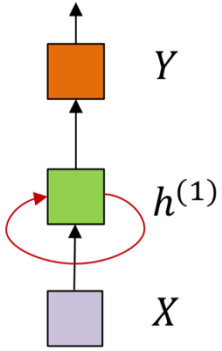
1. Estáticos:
   - Son números únicos que representan las palabras. Estos pueden ser valores pre-entrenados de las palabras, que luego se podrían usar para entrenar otros modelos. Por ejemplo: Glove (Latent Semantic Analysis-LSA) y word2vec (Continuous Bag-of-Words-CBOW, Skip-Gram) son embeddings estáticos. Están previamente entrenados. Son modelos independientes del contexto, como word2vec o GloVe, generan una representación de cada palabra en el vocabulario.
2. Contextual:
   - Cada palabra en los documentos tiene una representación independiente y continua (vector), que depende de su uso
   - Por ejemplo: ELMo, BERT son representaciones continuas y se pueden usar en tareas de aprendizaje posteriores.
   - Estos modelos contextuales generan una representación de cada palabra basado en las otras palabras de la oración.

Redes neuronales recurrentes

- Veamos un ejemplo: RNN, para predecir caracteres.
- Supongamos que tenemos la la cadena de caracteres “amor”; y, luego de ingresar las primeras 3 letras: “a, m, o”, deseamos que la RNN prediga la siguiente letra: “r”.
- En este ejemplo el vocabulario tiene 4 letras: {a, m, o, r}, y el algoritmo fue entrenado, recordar que este vocabulario se denomina “Corpus”.
- Veamos cómo esta estructura se utiliza para predecir la siguiente letra en la palabra “amor”.



Redes neuronales recurrentes

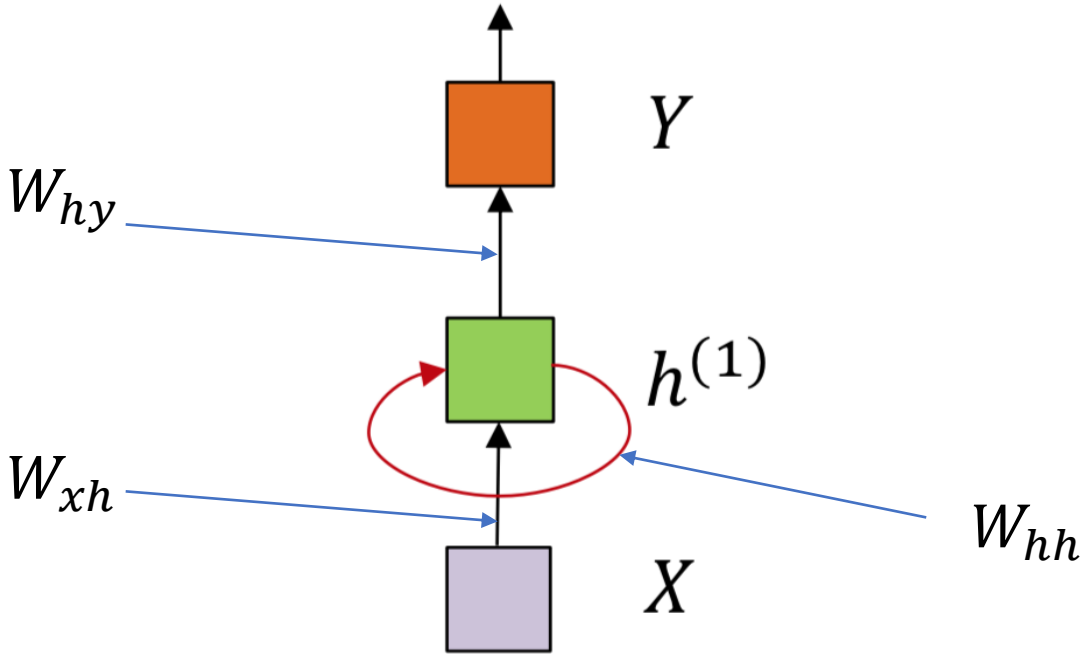


Procedimiento para el entrenamiento:

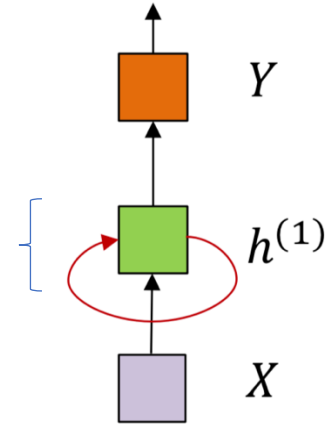
1. Ingresamos un símbolo a la vez (un time step); es decir, un X_t se suministra a la red.
2. Luego calculamos su estado actual utilizando la combinación de la entrada actual y el estado anterior, es decir calculamos h_t. Este h_t actual se convierte luego en h_{t-1} para el siguiente paso de tiempo.
3. Ejecutar tantos pasos de tiempo como el problema lo requiera y combinar la información de todos los estados anteriores.
4. Una vez que se completan todos los pasos de la secuencia, utilizamos el estado actual final para calcular la salida Y_t.
5. La salida se compara con la salida deseada y se calcula el error.
6. Luego, el error se propaga hacia atrás de la red para actualizar los pesos... Al final la red está entrenada.

Redes neuronales recurrentes

- Los pesos iniciales W_{xh}, W_{hh}, W_{hy} son generados aleatoriamente.
- No olvidar el bias



Redes neuronales recurrentes



- Supongamos una capa oculta con 3 neuronas de ancho
- Entonces necesitamos una matriz W_{xh} de dimensión: 3 (neuronas) x 4 (# de time-steps)

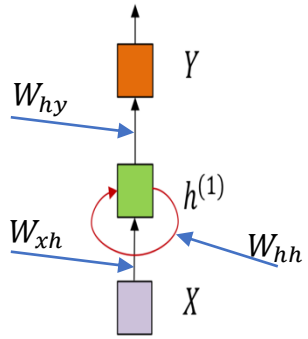
W_{xh}			
0.287027	0.84606	0.572392	0.486813
0.902874	0.871522	0.691079	0.18998
0.537524	0.09224	0.558159	0.491528

W_{hy}		
0.37168	0.974829459	0.830034886
0.39141	0.282585823	0.659835709
0.64985	0.09821557	0.334287084
0.91266	0.32581642	0.144630018

W_{hh}
0.427043

Bias
0.56700

Redes neuronales recurrentes

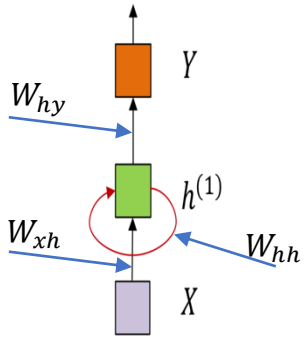


- Entonces: $h_0 = tanh(W_{xh}X_0 + (W_{hh}h_{0-1} + bias))$, es:

$$h_0 = \tanh\left\{\begin{bmatrix}0.287027359\\0.902874425\\0.537523791\end{bmatrix} + \begin{bmatrix}0.567001\\0.567001\\0.567001\end{bmatrix}\right\}$$

$$h_0 = \begin{bmatrix}0.69316804\\0.89955366\\0.80211840\end{bmatrix}$$

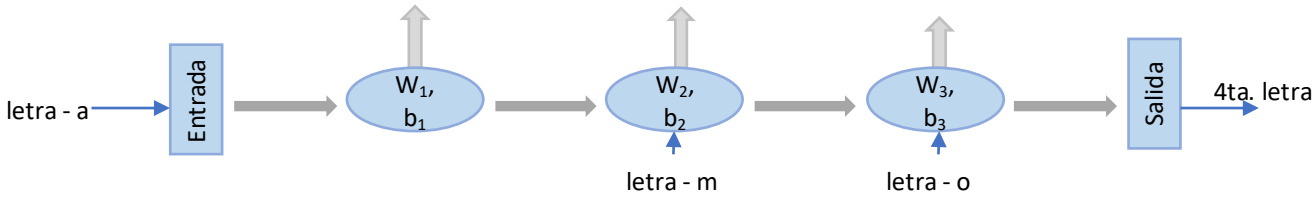
Redes neuronales recurrentes



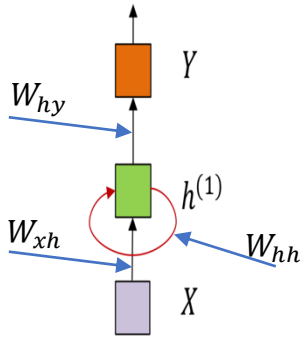
- Entonces el cálculo de $h_1 = tanh(W_{hh}h_0 + W_{xh}X_1)$, es:

$$h_1 = \tanh\left\{\begin{bmatrix}0.863013\\0.951149\\0.90954\end{bmatrix} + \begin{bmatrix}0.846060\\0.871522\\0.092240\end{bmatrix}\right\}$$

$$h_1 = \begin{bmatrix}0.93653372\\0.94910403\\0.76234057\end{bmatrix}$$



Redes neuronales recurrentes



- Finalmente, calculamos la probabilidad de las letras en el vocabulario, por ejemplo, aplicando la función de umbral softmax.
- La letra en particular que el sistema recomendaría, en nuestro ejemplo, después de “m”, sería la letra que obtenga la mayor probabilidad, luego de aplicar softmax.
- En nuestro ejemplo:

$$Y_t = \text{Softmax}$$

z_1
1.90607732
1.13779113
0.95666016
1.27422602

$=$

Probabilidades
0.419748
0.194682
0.162429
0.223141

Redes neuronales recurrentes

- Estas probabilidades lo que nos dicen es que la letra después de ”m" debe ser “a”, que tiene la probabilidad más alta.
- La predicción no es la mejor, puesto que apenas hemos empezado a entrenar la red.
- Acabamos de mostrarle dos letras. Así que prácticamente no ha aprendido nada todavía.
- Una vez que concluya la última letra “r”, cómo se actualizan los pesos?
- Como en los modelos anteriores, estas redes “aprenden” también a través de backpropagation.
- En RNN se lo llama “Back Propagation Through Time” (BPTT).